NAME: MADHUMITHA. R.V

STD: ~~DIV:~~ CSD - III yr ROLL NO.: 241701027

SUBJECT: GRAPHICS & MULTIMEDIA OBSERVATION.

INDEX

SR. NO.	DATE	TITLE	PAGE NO.	TEACHER'S SIGN
1.	21/07/26	Bresenham's line - Drawing Algorithm.	1	
2.	21/7/26	Bresenham's circle Algorithm	3	
3.	21/7/26	Bresenham's ellipse Algorithm	5	
4.	4/8/26	DDA algorithm using line Drawing	9	
5.	4/8/26	House - Drawing using Bresenham's algorithm	13	
6.	11/8/26	Rectangle using Bresenham's algorithm	19	
7.	11/8/26	Traffic signal using Bresenham's algorithm	21	
8.	25/8/26	Smiling face with eyes, nose, mouth using Bresenham's algorithm.	23	
9.	25/8/26	Olympic logo using ellipse	25	
10.	25/8/26	Butterfly using ellipse	27	
11.	8/9/26	Implementation of scan conversion of polygon using flood fill algorithm	29	
12.	22/9/26	Basic transformations for 2D shapes in Triangle	31	
13.	22/9/26	Basic Transformations for 2D shapes in Rectangle	33	
14.	22/9/26	Sutherland Hodgman Polygon clipping	35	
15.	29/9/26	3D transformations for cube	37	
16.	29/9/26	3D transformations for cone	39	
17.	10/10/26	Rendering 3D objects using polygon & shading Technique	41	
18.	10/10/26	Multimedia app integrates sound, Image & video in UI	43	
19.	10/10/26	Capture Video/Audio from Webcam or microphone & display on multimedia	45	

— Completed

12/10/26

RAJALAKSHMI
ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME .....MADHUMITHA.R.V.....

ACADEMIC YEAR ..2026-2027.. SEMESTER ....V.....BRANCH....B.E.-CSD

UNIVERSITY REGISTER No. 211624701027

Certified that this is the bonafide record of work done by the above student in the

IT23E31
Graphics & Multimedia Laboratory during the year 2026 - 2027

12/6/26

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examiner

SCAN AnyScanner

EXP - 1

DATE : 21/07/26

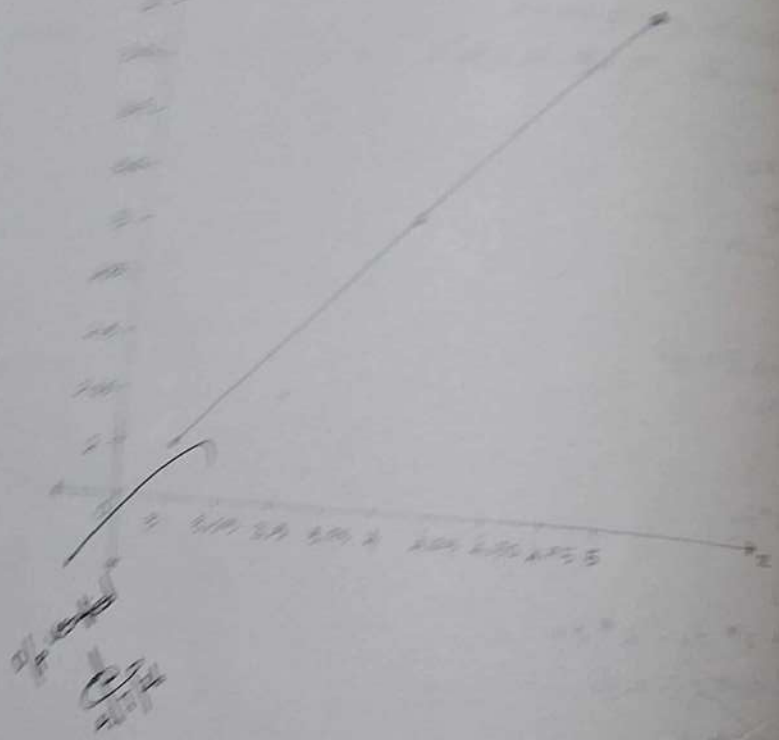
BRESENHAM'S LINE - DRAWING ALGORITHM FOR |m| < 1

AIM : To implement Bresenham's Line Drawing Algorithm in python and genrate the points of a straight line between two given co-ordinates.

PROGRAM :

```
import matplotlib.pyplot as plt
def bresenham_line (x1, y1, x2, y2):
points = []
dx = x2 - x1
dy = y2 - y1
P = 2* dy - dx
x = x1
y = y1
Points.append ((x,y))
while x < x2 :
x = x+1
if P<0 :
P = P + 2*dy
else :
y = y+1
P = P + 2* dy - 2 * dx
points. append ((x, y))
return points
x1 = int (input ("Enter x1 : "))
y1 = int (input ("Enter y1 : "))
x2 = int (input ("Enter x2 : "))
y2 = int (input ("Enter y2 : "))
points = bresenham_line (x1, y1, x2, y2)
print ("points genrated : ")
for p in points :
print (p)
```

```
x_values = [p[0] for p in points]
y_values = [p[1] for p in points]
plt.plot(x_values, y_values, marker = "o")
plt.title("Bresenham Line Drawing Algorithm (|m| > < 1)")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid()
plt.show()
```



21/7/26

RESULT :

The Bresenham Line Drawing Algorithm was successfully implemented. The program generated the intermediate pixel coordinates between the start and end point displayed.

EXP-2

DATE : 21/4/26

BRESENHAM'S CIRCLE ALGORITHM

AIM : To implement bresenham's circle algorithm in python and generate the points of a circle in the given co-ordinates.

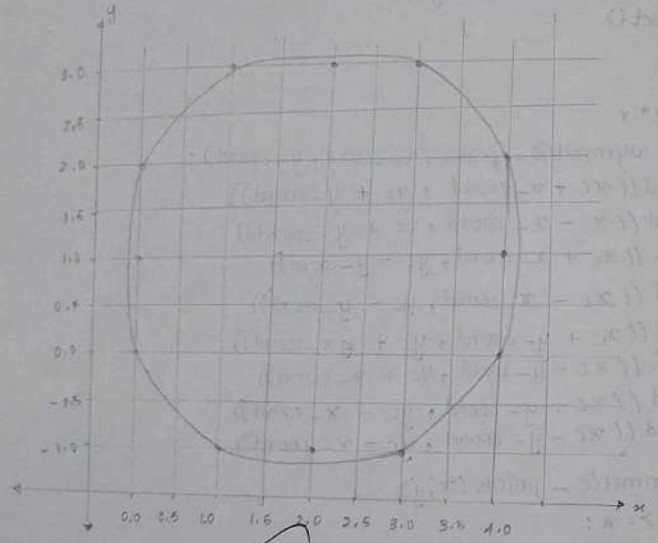
PROGRAM:

```
import matplotlib.pyplot as plt
def bresenham_circle (xc, yc, r):
points = set()
x = 0
y = r
d = 3 - 2 * r
def plot_symmetric_points (x_coord, y_coord):
points.add ((xc + x_coord , yc + y_coord))
points.add ((xc - x_coord , yc + y_coord))
points.add ((xc + x_coord, yc - y_coord))
points.add ((xc - x_coord, yc - y_coord))
points.add ((xc + y_coord , yc + x_coord))
points.add ((xc - y_coord , yc + x_coord))
points.add ((xc + y_coord , yc - x_coord))
points.add ((xc - y_coord , yc - x_coord))
plot_symmetric_points (x,y)
while y >= x:
    x += 1
    if d > 0:
    y -= 1
    d = d + 4 * (x - y) + 10
  else:
    d = d + 4 * x + 6
    plot_symmetric_points (x, y)
  return sorted (list (points))
xc = int (input ("Enter the x-coordinate of the centre (xc): "))
yc = int (input ("Enter the y-coordinate of the centre (yc): "))
r = int (input ("Enter the radius (r): "))
circle_points = bresenham_circle (xc, yc, r)
print ("\n Points generated for the circle : ")
for p in circle_points:
  print (p)
```

OUTPUT:

x - co-ord of the centre (xc) : 2
y - co-ord of the centre (yc) : 1
Enter the radius (r) : 2

points : (0,0), (0,1), (0,2), (1,-1), (1,3), (2,-1), (2,3), (3,-1), (3,3), (4,0), (4,1), (4,2).



o/p verified

```
x_coords = [p[0] for p in circle_points]
y_coords = [p[1] for p in circle_points]

plt.figure(figsize=(8, 8))
plt.plot(x_coords, y_coords, marker='o', linestyle='')
plt.title("Bresenham's Circle Drawing Algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.gca().set_aspect('equal', adjustable='box')
plt.show()
```

RESULT:

The Bresenham circle Algorithm was successfully implemented, The program generated the intermediate pixel coordinates between is displayed.

DATE : 21/07/26

EXP - 3

BRESENHAM'S ELLIPSE ALGORITHM

AIM : To implement brensenham's ellipse algorithm in python, and generate the points of ellipse in a given co-ordinates.

PROGRAM :

```
import matplotlib.pyplot as plt
def bresenham_ellipse (xc, yc, rx, ry):
x = 0
y = ry
rx2 = rx * rx
ry2 = ry * ry
p1 = ry2 - (rx2 * ry) + (0.25 * rx2)
points = []
while (2 * ry2 * x) < (2 * rx2 * y):
  points.append ((x, y))
if p1 < 0:
  x = x + 1
  p1 = p1 + (2 * ry2 * x) + ry2
else :
  x = x + 1
  y = y - 1
  p1 = p1 + (2 * ry2 * x) - (2 * rx2 * y) + ry2
p2 = (ry2 * (x + 0.5) * (x + 0.5)) + \(rx2 * (y - 1) * (y - 1))
  - \(rx2 * ry2)
while y >= 0:
  points.append ((x, y))
if p2 > 0:
  y = y - 1
  p2 = p2 - (2 * rx2 * y) + rx2
else :
  x = x + 1
  y = y - 1
p2 = p2 + (2 * ry2 * x) - \(2 * rx2 * y) + rx2
```

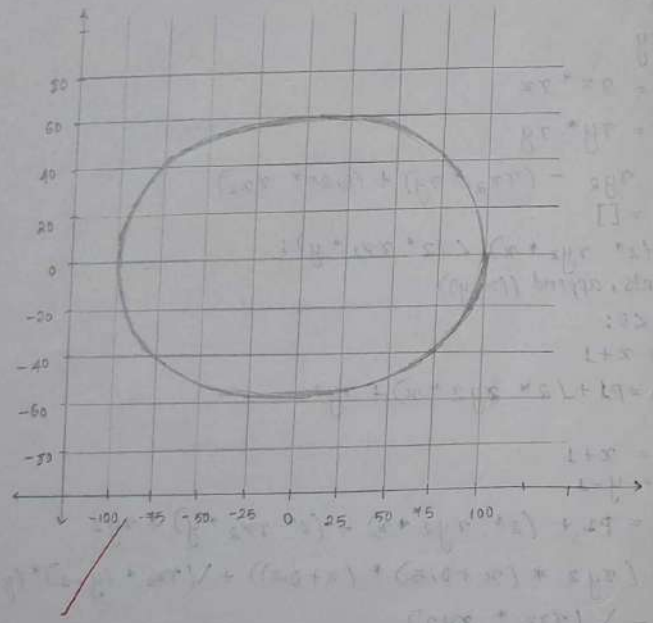
OUTPUT:

$x_c = 0$

$y_c = 0$

$r_x = 100$

$r_y = 50$




```
ellipse_points = []
for x, y in points:
    ellipse_points.append((xc + x, yc + y))
    ellipse_points.append((xc - x, yc + y))
    ellipse_points.append((xc + x, yc - y))
    ellipse_points.append((xc - x, yc - y))
return ellipse_points
points = bresenham_ellipse(xc, yc, rx, ry)
x_values = [p[0] for p in points]
y_values = [p[1] for p in points]
plt.scatter(x_values, y_values, s=2)
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.title("Bresenham's Ellipse Drawing Algorithm")
plt.axis("equal")
plt.grid()
plt.show()
```

28/11/20

RESULT:

The Bresenham's ellipse Algorithm was successfully implemented, the program generated the intermediate pixel coordinates btw is displayed.

EXP - 4

DATE : 04/8/26

DDA LINE DRAWING ALGORITHM

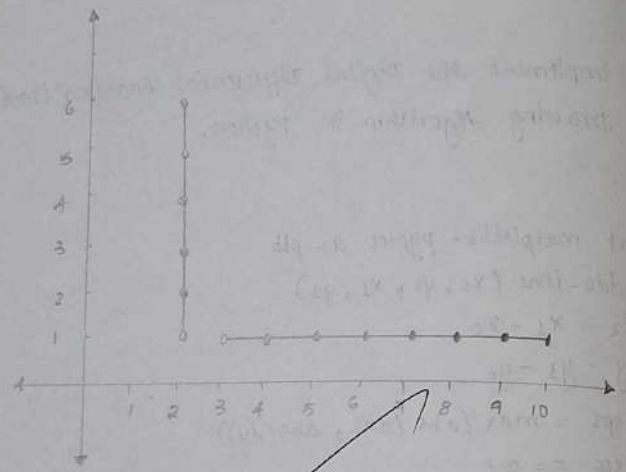
AIM: To Implement the Digital Differential Analyzer (DDA) Line Drawing Algorithm in Python.

PROGRAM:

```
import matplotlib.pyplot as plt
def dda_line (x0, y0, x1, y1)
  dx = x1 - x0
  dy = y1 - y0
  steps = max (abs (dx), abs(dy))
if steps == 0 :
    return [(x0, y0)]
  x_increment = dx/steps
  y_increment = dy/steps
  x = x0
  y = y0
  points = []
  points. append ((round(x), round(y)))
for _ in range (int(steps)):
    x+ = x_increment
    y+ = y_increment
    points. append ((round(x), round (y)))
  return points.

x_points, y_points = zip (* dda_line (2,3, 10,8))
 x_points 2 , y_points 2 = zip (* dda_line( 1, 1, 1,5))
plt. figure (figsize = (8,6))
plt. plot (x_points2, y_points2, marker = 'o',
      linestyle = '-', color = 'r')
plt. title ('DDA line Drawing algorithm (Vertical line)')
```

OUTPUT:



plt. xlabel ('x-axis')
plt. ylabel ('y-axis')
plt grid (True)
plt. xticks (range (min (x-points)-1, max (x-points 2)+2))
plt. gca(), set_aspect ('equal', adjustable = 'box')
plt. show()

4/8/24

RESULT: The line using the Digital Differential Analyzer has been successfully implemented.

EXP-5

DATE - 4/3/26

HOUSE DRAWING USING THE BRESENHAM'S ALGORITHM

AIM: To implement Bresenham's House drawing algorithm in python with the given co-ordinates.

PROGRAM:

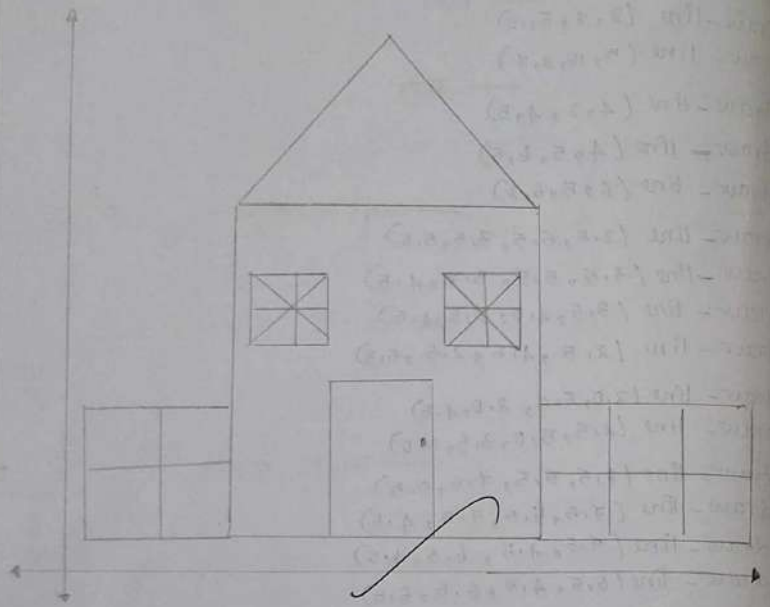
```
import matplotlib.pyplot as plt
def bresenham_line (x0, y0, x1, y1):
  dx = abs (x1 - x0)
  dy = abs (y1 - y0)
  Sx = 1 if x0 < x1 else -1
  Sy = 1 if y0 < y1 else -1
  err = dx - dy
  points = []
while True:
  points. append ((x0, y0))
  if x0 == x1 and y0 == y1:
    break
e2 = 2* err
if e2 > - dy:
    err - = dy
      x0 + = Sx
if e2
    err + = dx
      y0 + = sy
    return points

def draw_line (x0, y0, x1, y1)
  points = bresenham_line (x0, y0, x1, y1)
  x = [p[0] for p in points]
  y = [p[1] for p in points]
  plt. plot (x, y, '-s', color = 'purple', line width = 2,
                                    marksize = 5)
```

```
def draw_house():
    plt.figure(figsize=(10,8))
    ---- House walls
    draw_line(2,2,2,7)
    draw_line(8,2,8,7)
    draw_line(8,7,2,7)
    draw_line(2,7,2,2)
    ---- Roof
    draw_line(2,7,5,10)
    draw_line(5,10,8,7)
    ---- Door
    draw_line(4,2,4,5)
    draw_line(4,5,6,5)
    draw_line(6,5,6,2)
    ---- Left window
    draw_line(2.5,5.5,3.5,5.5)
    draw_line(3.5,5.5,3.5,4.5)
    draw_line(3.5,4.5,2.5,4.5)
    draw_line(2.5,4.5,2.5,5.5)

    draw_line(3.0,5.5,3.0,4.5)
    draw_line(2.5,5.0,3.5,5.0)
    ---- Right window
    draw_line(6.5,5.5,7.5,5.5)
    draw_line(7.5,5.5,7.5,4.5)
    draw_line(7.5,4.5,6.5,4.5)
    draw_line(6.5,4.5,6.5,5.5)
    draw_line(7.0,5.5,7.0,4.5)
    draw_line(6.5,5.0,7.5,5.0)
    ---- Ground
    draw_line(0,2,15,2)
    ---- left fence
    for x in range(0,2):
        draw_line(x,2,x,4)
    draw_line(0,4,2,4)
    draw_line(0,3,2,3)
    ---- Right fence
    for x in range(9,16):
        draw_line(x,2,x,4)
    draw_line(9,4,15,4)
    draw_line(9,3,15,3)
    ---- Door Handle
    draw_line(5.8,3.5,5.8,3.5)
```

OUTPUT:



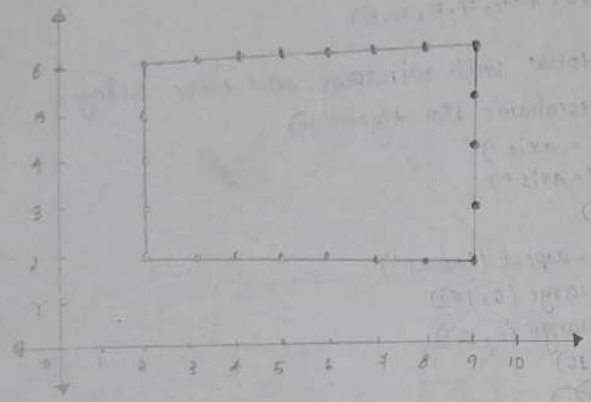

```
draw_line (2.5, 5.5, 3.5, 4.5)
draw_line (2.5, 4.5, 3.5, 5.5)
draw_line (6.5, 5.5, 7.5, 4.5)
draw_line (6.5, 4.5, 7.5, 5.5)

plt.title ("House with windows and Fence using
            Bresenham's line Algorithm)
plt.xlabel ("X-axis")
plt.ylabel ("Y-axis")
plt.grid (True)
plt.gca().set_aspect ('equal')
plt.xticks (range (0, 17))
plt.yticks (range (0, 12))
plt.xlim (-1, 16)
plt.ylim (-1, 11)
plt.show()
drawHouse()
```

11/8/24

RESULT : The House with windows and fence using the Bresenham's algorithm has been successfully implemented.

OUTPUT :



EXP-6

DATE : 11/7/26

RECTANGLE USING BRESENHAM'S ALGORITHM

Given : width = 8, height = 5, vertex (2,2)

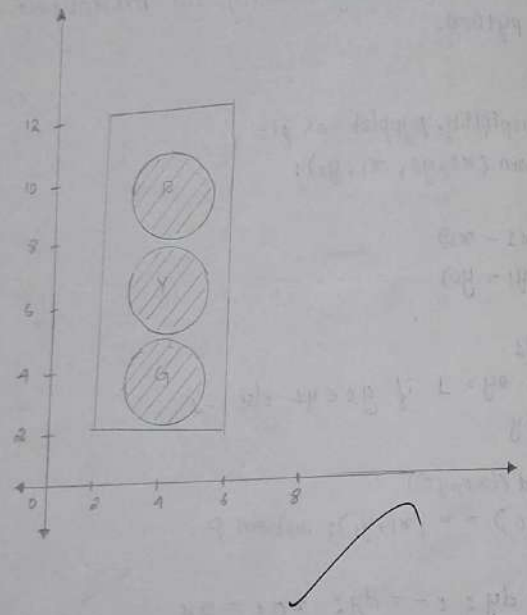
AIM : To implement the rectangle using the bresenham's algorithm in python.

PROGRAM :

```
import matplotlib.pyplot as plt
def bresenham(x0, y0, x1, y1):
P = []
dx = abs(x1 - x0)
dy = abs(y1 - y0)
sx = 1
if x0 < x1
else -1 ; sy = 1 if y0 < y1 else -1
e = dx - dy
while 1 :
  p. append ((x0, y0))
  if (x0, y0) == (x1, y1): return P
  e2 = 2*e
  if e2 > -dy : e -= dy;  x0 += sx
  if e2 < dx ;  e += dx ; y0 += sy
x, y, w, h = 2, 2, 8, 5
  xe, ye = x + w - 1 , y + h - 1
for a, b, c, d in [(x, y, xe, y), (x, ye, xe, ye), (x, y, x, ye),
                   (xe, y, xe, ye)]:
p = bresenham(a, b, c, d)
 plt. plot (* zip (* p), 'bo-')
plt. grid()
plt. axis ('equal')
plt. show()
```

RESULT : The rectangle is drawn successfully using bresenham's algorithm.

OUTPUT :-



EXP - 7

DATE : 11/2/26

TRAFFIC SIGNAL USING BRESENHAM'S ALGORITHM

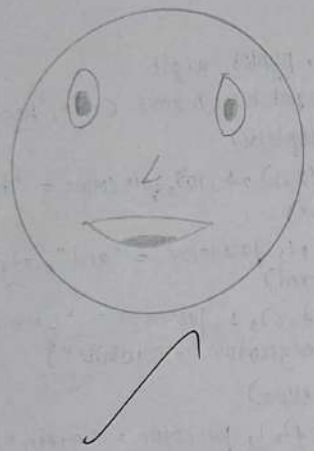
AIM : To implement the program to draw the traffic signal using bresenham's Algorithm.

PROGRAM :

```
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Rectangle
fig, ax = plt.subplots()
box = Rectangle ((2,1), 4, 10, facecolor = "black")
ax.add_patch (box)
red = Circle ((4,2), 1, facecolor = "red", edgecolor = "white")
ax.add_patch (red)
yellow = Circle ((4,6), 1, facecolor = "yellow",
                 edgecolor = "white")
ax.add_patch (yellow)
green = Circle ((4,4), 1, facecolor = "green", edgecolour
                                = "white")
ax.add_patch (green)
ax.set_xlim (0,8)
ax.set_ylim (0,12)
ax.set_aspect ("equal")
ax.grid (True)
plt.title ("Traffic signal")
plt.show()
```

RESULT : The program for the traffic signal using Bresenham's Algorithm using the bresenhams algorithm.

OUTPUT:



EXP-8

DATE : 25/3/24

SMILING FACE WITH EYES, NOSE, MOUTH USING BRESENHAM'S ALGORITHM

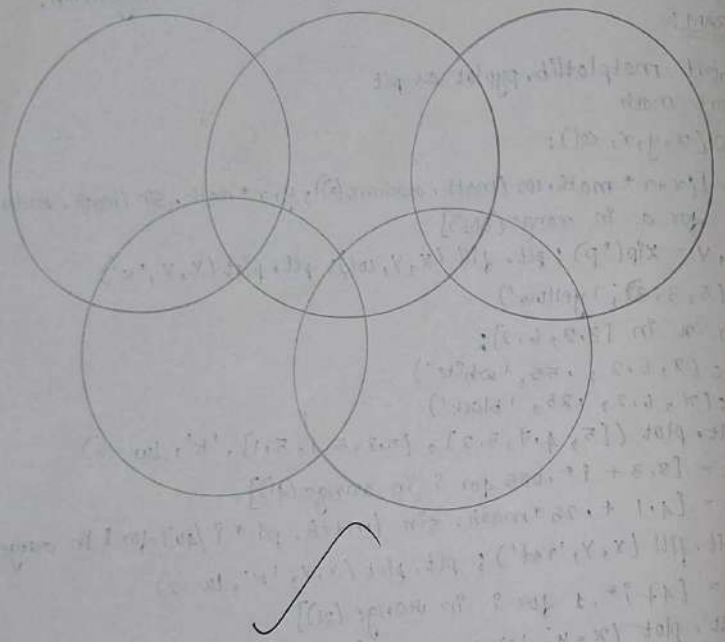
AIM: To Implement the program to draw a smiling face with eyes, nose, mouth using Bresenham's Algorithm.

PROGRAM:

```
import matplotlib.pyplot as plt
import math
def c(x,y,r, col):
p = [(x+r* math.cos(math.radians(a)), y+r* math.sin(math.radians(a)
    for a in range(361)]
 x, y = zip(*p); plt.fill(x, y, col); plt.plot(x, y, 'k')
 c(5, 3, 3, 'yellow')
for x in [3.8, 6.2]:
  c(x, 6.2, .55, 'white')
  c(x, 6.2, .25, 'black')
 plt.plot([5, 4.7, 5.2], [5.8, 5.1, 5.1], 'k', lw=2)
 x = [3.3 + i*.035 for i in range(41)]
 t = [4.1 + .25*math.sin(math.pi * i/40) for i in range(41)]
 plt.fill(x, y, 'red'); plt.plot(x, y, 'k', lw=2)
 x = [4+i*.1 for i in range(21)]
 plt.plot(x, y, 'pink', lw=4)
 plt.axis('equal')
 plt.xlim(1,9); plt.ylim(1,9)
 plt.grid()
 plt.title("smiling face")
 plt.show()
```

RESULT: The program to implement the smiling face has been done successfully.

OUTPUT:



EXP - 9

DATE : 25/8/26

OLYMPIC LOGO USING ELLIPSE

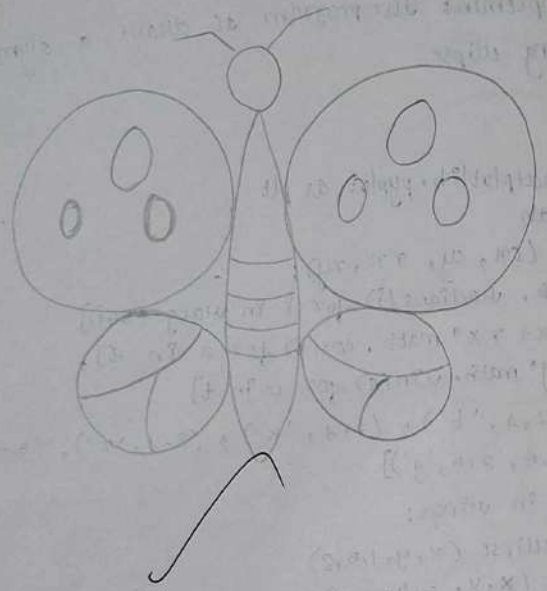
AIM : To implement the program to draws a olympic logo using ellipse

PROGRAM :

```
import matplotlib.pyplot as plt
import math
def ellipse (cx, cy, rx, ry):
t = [math.radians(i) for i in range(361)]
return [cx + rx * math.cos(a) for a in t],
  [cy + ry * math.sin(a) for a in t]
rings = [(2,4,'b'), (5,4,'k'), (8,4,'r'), (3.5,2.5,'y')
    (6.5, 2.5,'g')]
for x,y,c in rings:
    x, Y = ellipse (x, y, 1.5,8)
    plt.plot (x,Y, c,lw = 3)
plt.axhline (0, color = 'k')
plt.axvline (0, color = 'k')
plt.grid()
plt.xlim (0,10)
plt.ylim(0.6)
plt.xticks (range(11))
plt.yticks (range(7))
plt.axis ('equal')
plt.title ("olympic logo - DDA")
plt.show()
```

RESULT : The program to implement the olympic logo using ellipse is done successfully.

OUTPUT:



EXP-10

DATE - 25/8/24

BUTTERFLY USING ELLIPSE

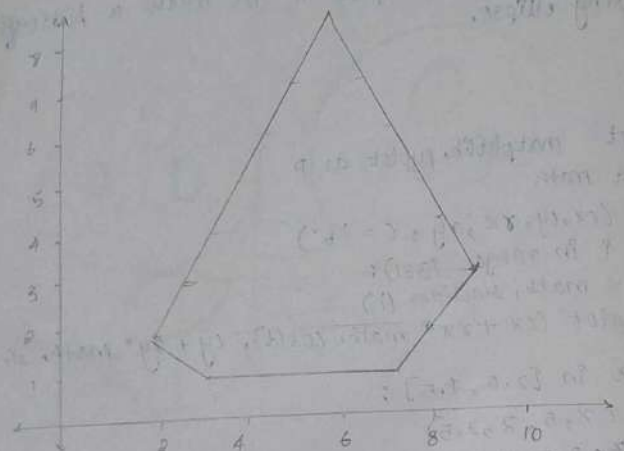
AIM: To implement the program to draw a butterfly using ellipse.

PROGRAM:

```
import matplotlib.pyplot as p
import math
def e (cx,cy,rx,ry, c = 'b')
for i in range (361):
  t = math.radians (i)
  p.plot (cx + rx* math.cos(t), cy + ry* math.sin(t),c)
for x in [2.5, 7.5]:
  e (x, 5, 2, 2.5)
e (x, 2.5, 1.5, 1.5)
e (5, 4.5, 3, 2.5, 'k')
  e (5, 4, .5, .5, 'k')
p.plot ([4.3, 4, 3.5], [7.1, 8, 8.3], 'k')
p.plot ([5.2, 6, 6.5], [7.4, 8, 8.3], 'k')
p.axis ('equal')
p.grid ()
p.show ()
```

RESULT: The program to draw a buttoyly has been successfully done.

OUTPUT:



EXP-11

DATE: 8/9/26

Pg-29

IMPLEMENTATION SCAN CONVERSION OF POLYGON AND USE FLOOD FILL ALGORITHM

AIM: To implement a scan conversion for a polygon and use flood fill algorithm in python.

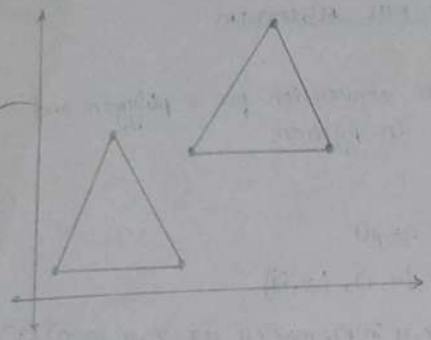
PROGRAM:

```
import matplotlib.pyplot as plt
P = [(2,2), (5,8), (9,3), (7,1), (3,1)]
def scan_fill(p):
    ys = range(min(y for x,y in p), max(y for x,y in p)+1)
    for y in ys:
        x = []
        for i in range(len(p)):
            x1, y1 = p[i]
            x2, y2 = p[(i+1)%len(p)]
            if y1 != y2 and min(y1,y2) <= y < max(y1,y2):
                x.append(x1 + (y-y1)* (x2-x1)/(y2-y1))
        x.sort()
        for i in range(0, len(x), 2):
            if i+1 < len(x):
                plt.plot(x[i:i+2], [y,y], 'b-')
x, y = zip(*(p + [p[0]]))
plt.plot(x, y, 'k-', linewidth=2)
scan_fill(p)
plt.fill(x, y, color='orange', alpha=0.7)
plt.axis('equal')
plt.title("scan conversion and Flood fill")
plt.show()
```

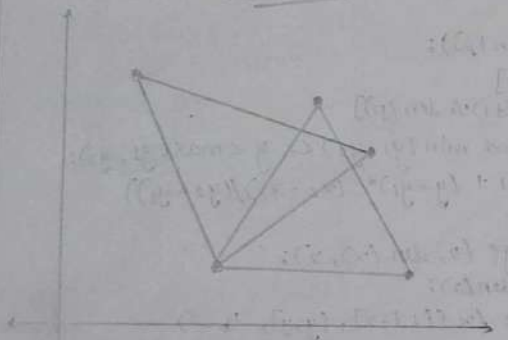
RESULT: The program for the scan conversion of polygon and use flood fill algorithm is done successfully.

OUTPUT:

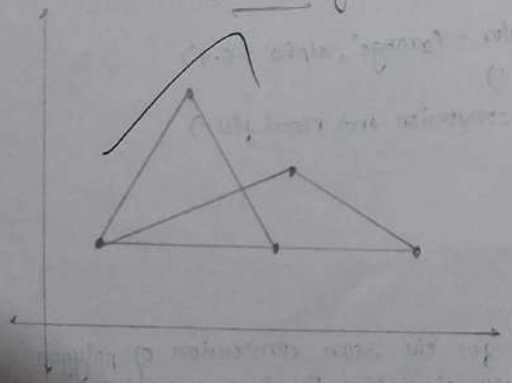
2D Translation



2D Rotation



2D scaling



EXP- 12

BASIC TRANSFORMATIONS FOR 2D SHAPES FOR TRIANGLE

AIM: To implement the program for the basic transformations for 2D shapes to draw triangle in python.

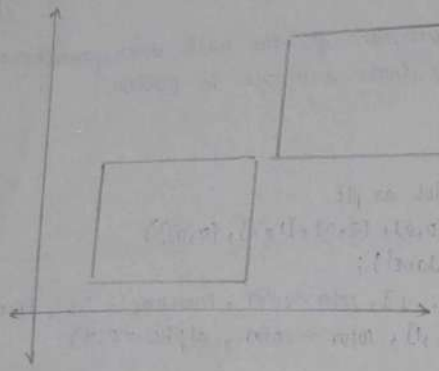
PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
shape = np.array([[0,0], [2,0], [1,2], [0,0]])
def plot(ax, s, color, label):
    ax.plot(s[:,0], s[:,1], color=color, marker='o', label)
    ax.fill(s[:,0], s[:,1], color=color, alpha=0.3)
    ax.grid()
    ax.axis('equal')
    ax.legend()
def translate(s, dx, dy):
    return s + [dx, dy]
def rotate(s, angle):
    a = np.radians(angle), R = np.array([[np.cos(a), -np.sin(a)],
                                         [np.sin(a), np.cos(a)]])
    return s @ R.T
def scale(s, sx, sy):
    return s * [sx, sy]
(i). t = translate(shape, 2, 1)
fig, ax = plt.subplots()
plot(ax, shape, 'blue', 'original'), plot(ax, t, 'red', 'Translated')
ax.set_title('2D Translation')
plt.show()
(ii). r = rotate(shape, 45)
fig, ax = plt.subplots()
plot(ax, r, 'blue', 'original'), plot(ax, r, 'green', 'rotated 45°')
ax.set_title('2D Rotation')
plt.show()
(iii). s = scale(shape, 2, 0.5)
fig, ax = plt.subplots()
plot(ax, shape, 'blue', 'original'), plot(ax, s, 'purple', 'scaled')
ax.set_title('2D scaling')
plt.show()
```

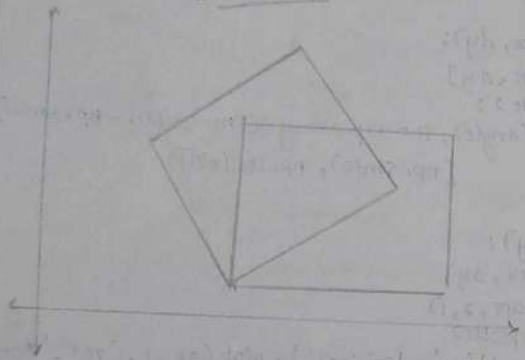
RESULT: The program to draw the triangle for the basic transformations in python.

OUTPUT:

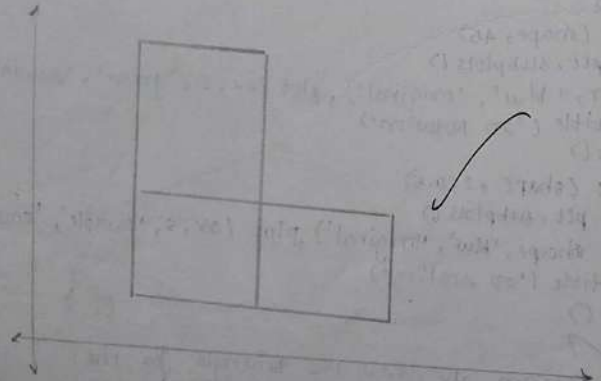
2D Translation



2D Rotation



2D - Scaling



EXP-13 BASIC TRANSFORMATIONS FOR 2D SHAPES FOR RECTANGLE

DATE - 22/9/26

Pg-33

AIM: To implement the basic transformations for 2D shapes to draw rectangle in python.

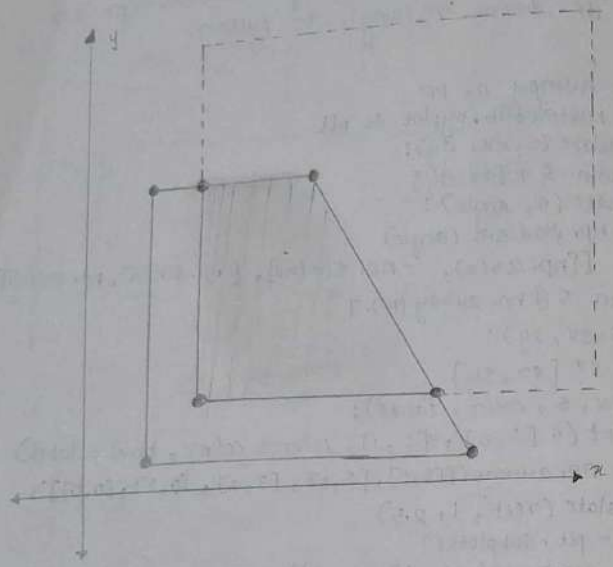
PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
def translate(s, dx, dy):
    return s + [dx, dy]
def rotate(s, angle):
    a = np.radians(angle)
    R = [[np.cos(a), -np.sin(a)], [np.sin(a), np.cos(a)]]
    return s @ np.array(R).T
def scale(s, sx, sy):
    return s * [sx, sy]
def plot(ax, s, color, label):
    ax.plot(s[:,0], s[:,1], color = color, label = label)
rect = np.array([[0,0],[3,0],[3,2],[0,2],[0,0]])
t = translate(rect, 1, 2.5)
fig, ax = plt.subplots()
plot(ax, rect, 'blue', 'Original'), plot(ax, t, 'red', 'Tran
ax.set_title('2D Translation')
ax.grid() ax.axis('equal') ax.legend()
plt.show()
r = rotate(rect, 30)
fig, ax = plt.subplots()
plot(ax, rect, 'blue', 'Original'), plot(ax, r, 'green', 'Rot
ax.set_title('2D Rotation')
ax.grid() ax.axis('equal') ax.legend()
plt.show()
s = scale(rect, 0.5, 1.5)
fig, ax = plt.subplots()
plot(ax, rect, 'blue', 'Original'), plot(ax, s, 'purple', 'Scal
ax.set_title('2D scaling')
ax.grid() ax.axis('equal') ax.legend()
plt.show()
```

22/9/26

RESULT: The program to draw the rectangle and the basic transformations in python.

OUTPUT:



EXP-14 SUTHERLAND - HODGMAN POLYGON CLIPPING ALGORITHM

DATE: 22/9/26
Pg-35

AIM: To implement the algorithm to clip a subject polygon against a rectangular clipping window using python.

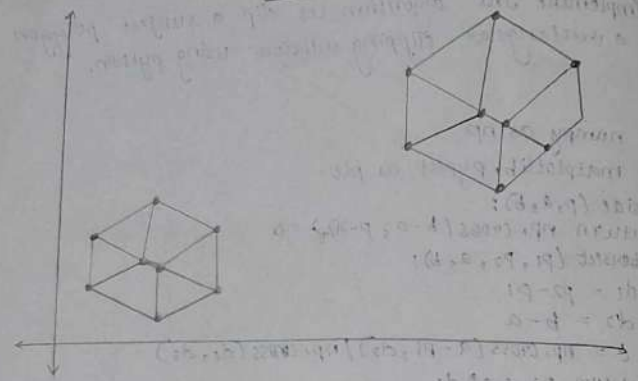
Program:

```
import numpy as np
import matplotlib.pyplot as plt
def inside(p,a,b):
    return np.cross(b-a, p-a) >= 0
def intersect(p1, p2, a, b):
    d1 = p2-p1
    d2 = b-a
    c = np.cross(a-p1, d2) / np.cross(d1, d2)
    return p2 + c*d1
def clip(subject, window):
    output = list(subject)
for i in range(len(window))-1):
    a, b = window[i], window[i+1]
    inp, output = output, []
for j in range(len(inp)):
    p1, p2 = inp[j-1], inp[j]
    in1, in2 = inside(p1, a, b), inside(p2, a, b)
if in2:
    if not in1:
        output.append(intersect(p1, p2, a, b))
elif in1:
    output.append(intersect(p1, p2, a, b)).
return np.array(output)
subject = np.array([[0.5,0.5], [3,0.5], [2,3], [0.5,3], [0.5,0.5]])
window = np.array([[1,1], [4,1], [4,4], [1,4], [1,1]])
result = clip(subject, window)
plt.plot(subject[:,0], subject[:,1], 'b-o', label='subject')
if len(result)>0:
    result = np.vstack([result, result[0]])
plt.title('polygon clipping'), plt.xlabel('x'), plt.ylabel('y')
plt.legend()
plt.show()
```

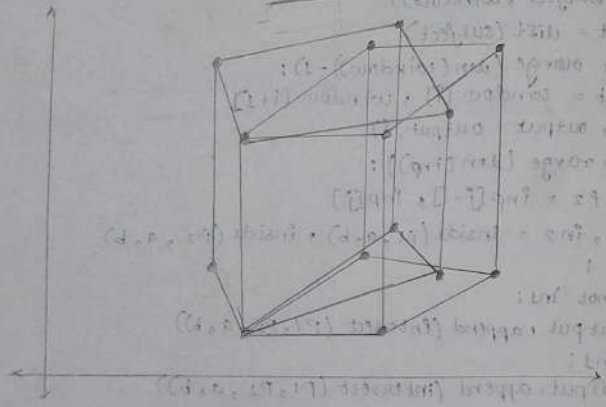
RESULT: The program for the Sutherland - hodgman polygon clipping algorithm is done.

OUTPUT:

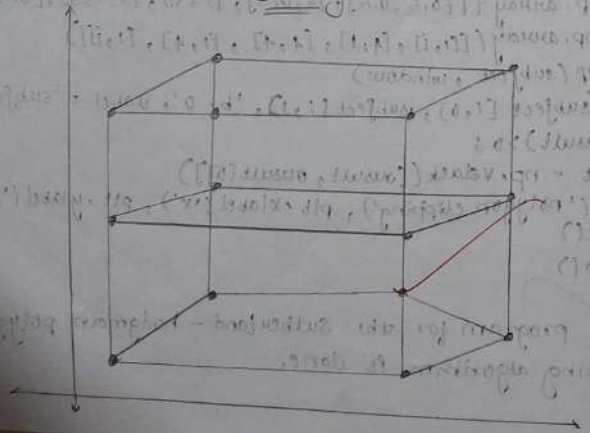
Translation



Rotation



Scaling



EXP-15

3D SHAPES TRANSFORMATIONS FOR CUBE

DATE: 29/9/26

Pg-37

AIM: To implement the transformations for 3D shapes for cube using the python.

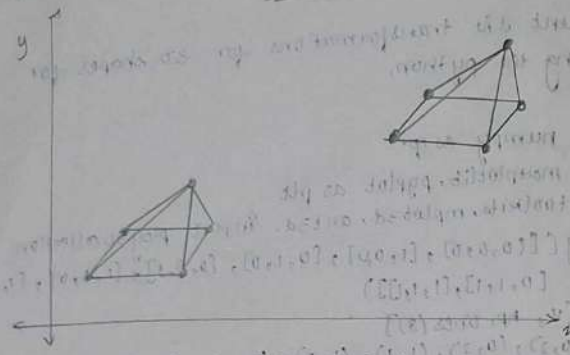
PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
V = np.array([[0,0,0],[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,0,1],
              [0,1,1],[1,1,1]])
H = np.c_[V, np.ones(8)]
E = [(0,1),(0,2),(0,3),(1,4),(1,5),(2,4),(2,6),(3,5),(3,6),(4,7),
     (5,7),(6,7)]
F = [[0,1,4,2],[3,5,7,6],[0,1,5,3],[2,4,7,6],[0,2,6,3],
     [1,4,7,5]]
def plot(T, title, color):
    w = (T @ H.T)[:, :3]
    fig = plt.figure(figsize=(8,6))
    ax = fig.add_subplot(111, projection='3d')
    for v, c in [(V,'blue'),(W,colour)]:
        for i,j in E:
            ax.plot(*zip(V[i],V[j]), color=c)
        ax.scatter(V[:,0],V[:,1],V[:,2],color=c)
    faces = [[V[k] for k in f] for f in F]
    ax.add_collection3d(Poly3DCollection(faces, alpha=2,
                        facecolor=c))
    ax.set(xlabel='x', ylabel='y', zlabel='z', title=title)
    plt.show()
T = np.array([[1,0,0,2],[0,1,0,2],[0,0,1,2],[0,0,0,1]])
a = np.pi/4
T = np.array([[np.cos(a),-np.sin(a),0,0],[np.sin(a),np.cos(a),0,0],
              [0,0,1,0],[0,0,0,1]])
T = np.diag([1.5,0.5,2,1]) plot(T)
```

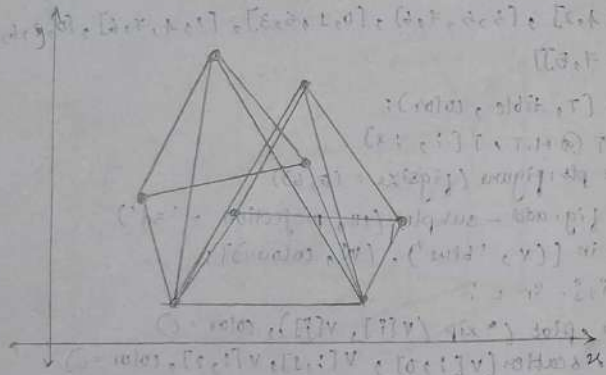
RESULT: The program for the 3D transformations for cube is done.

OUTPUT:

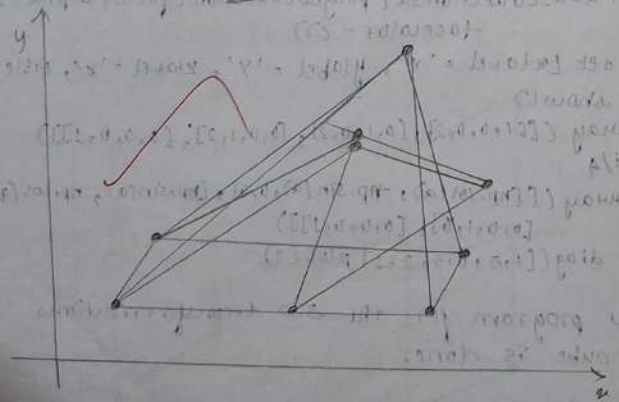
Translation



Rotation



Scaling



EXP-16

3D TRANSFORMATIONS FOR CONE

Date: 29/9/26

Pg-39

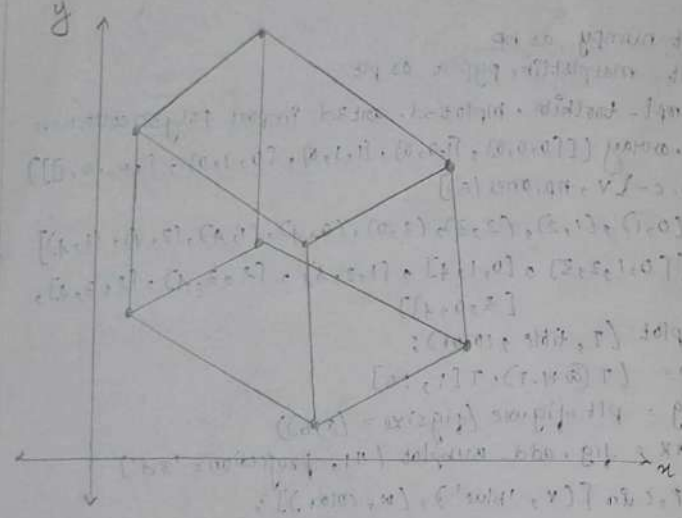
AIM: To implement the program for the 3D shapes transformation for cone.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
V = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],[.5,.5,1]])
H = np.c_[V, np.ones(5)]
E = [(0,1),(1,2),(2,3),(3,0),(0,4),(1,4),(2,4),(3,4)]
F = [[0,1,2,3],[0,1,4],[1,2,4],[2,3,4],[3,0,4]]
def plot(T, title, color):
    w = (T @ H.T).T[:, :3]
    fig = plt.figure(figsize=(8,6))
    ax = fig.add_subplot(111, projection='3d')
for P,c in [(V,'blue'),(w,color)]:
    for i,j in E:
        ax.add_collection3d(Poly3DCollection([[P[k] for k in f]
                for f in F], alpha=.2, facecolor=c))
ax.set(xlabel='x', ylabel='y', zlabel='z', title=title)
plt.show()
T = np.array([[1,0,0,2],[0,1,0,2],[0,0,1,2],[0,0,0,1]])
a = np.pi/4
T = np.array([[np.cos(a), -np.sin(a), 0, 0],
              [np.sin(a), np.cos(a), 0, 0], [0,0,1,0],
              [0,0,0,1]])
T = np.diag([1.5,.5,2,1])
```

RESULT: The program for 3D transformations for the cone is done implemented.

OUTPUT:



EXP-17

DATE - 10/10/26
PG - 41

RENDERING 3D OBJECTS USING POLYGONS AND SHADING TECHNIQUES

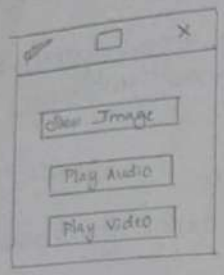
AIM: To create and render 3D objects using polygonal modelling and apply flat shading & Basic color techniques.

PROGRAM:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import numpy as np
vertices = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],
                     [0,0,1],[1,0,1],[1,1,1],[0,1,1]])
faces = [[vertices[j] for j in [0,1,2,3]],
         [vertices[j] for j in [4,5,6,7]],
         [vertices[j] for j in [0,1,5,4]]]
colors = ['red', 'blue', 'green', 'yellow', 'orange', 'purple']
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
cube = Poly3DCollection(faces, facecolors=colors, alpha=0.7)
ax.add_collection3d(cube)
ax.scatter(vertices[:,0], vertices[:,1], vertices[:,2],
           color='black')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.set_xlim(0,1)
ax.set_ylim(0,1)
plt.show()
```

RESULT: 3D polygonal objects were successfully created and rendered.

OUTPUT:



EXP-13 MULTIMEDIA APPLICATION INTEGRATES IMAGES, SOUND AND VIDEO IN UI

DATE-10/10/24
Pg-13

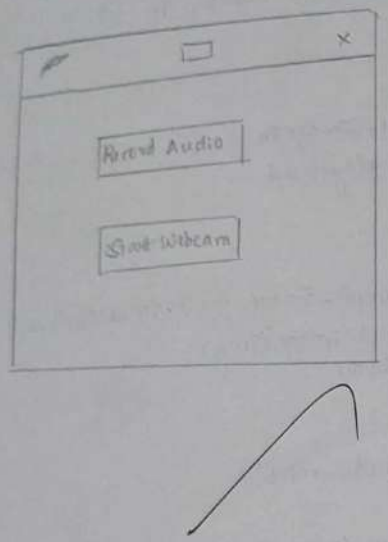
AIM: To develop a simple multimedia application that integrates images, video and audio. in UI.

PROGRAM:

```
import tkinter as tk
from PIL import Image, ImageTk
from playsound import playsound
import cv2
import threading
def image():
    img = Image.open("sample-image.jpg").resize((300,300))
    photo = ImageTk.PhotoImage(img)
    panel.config(image = photo)
    panel.image = photo
def audio():
    playsound("sample-audio.mp3")
def video():
    cap = cv2.VideoCapture("sample-video.mp4")
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow("video", frame)
        if cv2.waitKey(25) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()
root.title("Multimedia App")
panel.pack()
tk.Button(root, text = "show Image", text = "play Audio",
          text = "play video")
root.mainloop()
```

RESULT: A multimedia application was successfully created that integrates image, audio, and video into a UI.

OUTPUT:



EXP-19

DATE: 10/10/26

Pg-15

CAPTURE VIDEO/AUDIO FROM WEBCAM OR MICROPHONE AND DISPLAY IN MULTIMEDIA

AIM: To develop a program that captures live video, audio from the webcam and microphone.

PROGRAM:

```
import tkinter as tk
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import threading
def audio():
    fs = 44100
    print("Recording....")
    data = sd.rec(5 * fs, samplerate = fs, channels = 2)
    sd.wait()
    write("recorded_audio.wav", fs, data)
    print("Audio saved!")
def video():
    cap = cv2.VideoCapture(0)
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow("webcam", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()
root.title("Audio and Video Capture").
tk.Button(root, text = "Record Video" & "start webcam")
root.mainloop()
```

RESULT: Live video and audio were successfully captured using webcam & microphone.

Completed